

Modül 6

Lifecycle & Directives

Component Yaşam Döngüsü ve Direktifler

Modül:	6 / 30
Ders Sayısı:	7 ders
Seviye:	Orta
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 6'ya Hoş Geldin

Bu modülde Angular'ın component yaşam döngüsünü ve direktifler sistemini işleyeceğiz. Modern Angular'da lifecycle yönetimi `effect()` ve yeni signal API'lar ile değişti — eski hook'lar hala var ama farklı bir bakış açısı gerek.

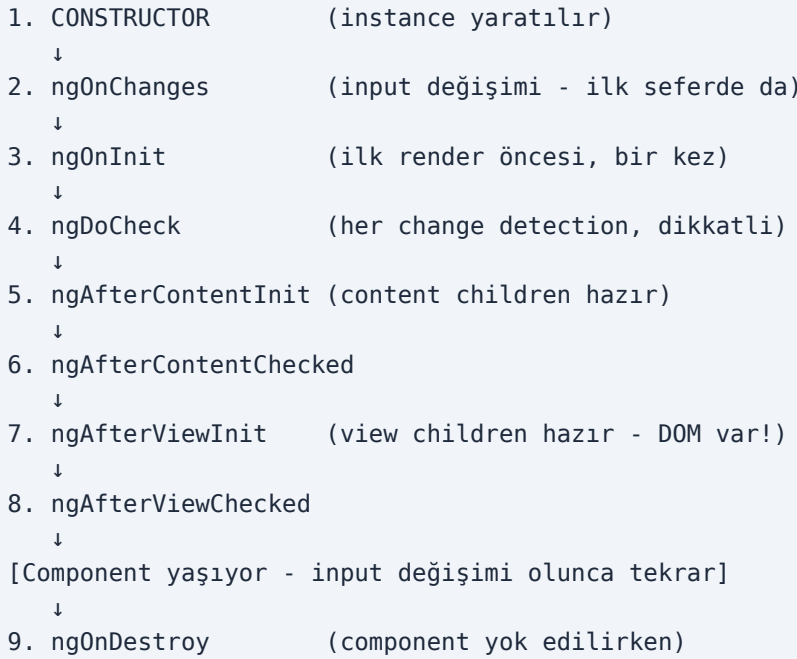
Bu modülün sonunda yapabileceklerin:

- ✓ Component lifecycle'ı tam anlayabilmek
- ✓ Lifecycle hook'ları doğru zamanda kullanabilmek
- ✓ Modern alternatiflerini (`effect`, `afterNextRender`) bilmek
- ✓ Custom attribute directive yazabilmek
- ✓ Structural directive yazabilmek (nadir ama bilmek lazım)
- ✓ Built-in directive'leri (`NgClass`, `NgStyle`, `NgIf` vs.) modern alternatifleri ile
- ✓ `@defer` ile lazy loading yapabilmek
- ✓ Host binding ve host listener kullanabilmek

Component Lifecycle — Genel Bakış

Bir component doğar, yaşar ve ölür. Bu yaşam döngüsünün aşamalarını ve Angular'ın hangi noktalarda hook sağladığını öğrenelim.

□ Component Yaşam Döngüsü Diyagramı



□ Sık Kullanılan 3 Hook

Pratikte 3 hook hayatın %95'ini oluşturur:

- **ngOnInit:** Component hazır olduğunda bir kez çalışır. Init işleri için.
- **ngOnChanges:** @Input değiştiğinde çalışır. (Modern Angular'da input() ile gereksiz hale geldi)
- **ngOnDestroy:** Component yok olurken çalışır. Cleanup için.

⚡ Modern Angular — Çoğu Hook Gereksiz!

Modern Angular'da signal'lar ve effect() sayesinde lifecycle hook'ların büyük çoğunluğu gereksiz. İşte değişim:

Eski Yol (Hooks)	Modern Yol (Signals)	Açıklama
ngOnChanges	effect() + input()	Input değişimi izlemek

ngDoCheck	computed()	Türemiş değer
ngOnInit (HTTP)	resource()	Async data fetch
ngOnDestroy	DestroyRef.onDestroy()	Cleanup
ngAfterViewInit	afterNextRender()	DOM erişimi

ngOnInit ve constructor — Doğru Kullanım

constructor ve ngOnInit arasındaki farkı net bilmek lazım. Bazen constructor, bazen ngOnInit doğru.

□ Constructor — Ne Zaman?

Constructor: TypeScript class'ı yaratılırken çalışır. Henüz input'lar belli değil.

```
@Component({...})
export class UserComponent {
  // Modern: inject() ile DI
  private userService = inject(UserService);
  private route = inject(ActivatedRoute);

  // Signal init
  user = signal<User | null>(null);

  constructor() {
    // ✓ DI initialization
    // ✓ Signal initial values
    // ✓ effect() oluşturma

    effect(() => {
      console.log('User changed:', this.user());
    });

    // x Input okumaya çalışma — henüz yok!
    // x HTTP isteği yapmaya çalışma (input gerekebilir)
  }
}
```

□ ngOnInit — Ne Zaman?

ngOnInit: Input'lar set edildi, component görünmeye hazır. Bir kez çalışır.

```

@Component({...})
export class UserDetailsComponent implements OnInit {
  userId = input.required<string>();

  ngOnInit() {
    // ✓ Input'lar artık hazır
    console.log('User ID:', this.userId());

    // ✓ İlk HTTP request
    this.loadUser(this.userId());
  }
}

```

❑ Karşılaştırma

Konu	constructor	ngOnInit
Input erişimi	Hayır	Evet
Ne zaman çalışır?	Class yaratılırken	Input set edildikten sonra
Kaç kere?	Bir kez	Bir kez
DI inject	✓ İdeal	Çalışır ama geç
Effect oluşturma	✓ İdeal	Çalışır
HTTP isteği	✗ Input olmadan?	✓ İdeal

❑ Modern Pratik

Modern Angular'da input'a göre tepki vermek için effect kullan:

```

@Component({...})
export class UserDetailsComponent {
  userId = input.required<string>();
  user = signal<User | null>(null);

  constructor() {
    // userId değişince OTOMATİK yeniden yükle
    effect(() => {
      const id = this.userId();
      this.loadUser(id);
    });
  }

  // ngOnInit gerek YOK!
  private async loadUser(id: string) {
    const user = await fetch(`/api/users/${id}`).then(r => r.json());
    this.user.set(user);
  }
}

```

ngOnDestroy ve DestroyRef

Cleanup için. Modern Angular'da DestroyRef daha esnek ve fonksiyonel bir alternatif.

❏ Klasik ngOnDestroy

```
@Component({...})
export class MyComponent implements OnDestroy {
  private subscription?: Subscription;

  ngOnInit() {
    this.subscription = this.dataService.data$.subscribe(...);
  }

  ngOnDestroy() {
    this.subscription?.unsubscribe();
  }
}
```

❏ Modern: DestroyRef

DestroyRef ile inject et, onDestroy callback ekle:

```
import { Component, inject, DestroyRef } from '@angular/core';

@Component({...})
export class MyComponent {
  private destroyRef = inject(DestroyRef);

  constructor() {
    const sub = this.dataService.data$.subscribe(...);

    this.destroyRef.onDestroy(() => {
      sub.unsubscribe();
    });
  }
}
```

🔪 takeUntilDestroyed() — En Yaygın

RxJS observable'ları otomatik unsubscribe yapmak için takeUntilDestroyed() operatorü:

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

@Component({...})
export class MyComponent {
  constructor() {
    this.dataService.data$
      .pipe(takeUntilDestroyed()) // ← Otomatik cleanup
      .subscribe(data => {
        console.log(data);
      });

    // Component destroy olunca subscription otomatik temizlenir
  }
}
```

❏ Memory Leak Önleme

Modern Angular'da `takeUntilDestroyed()` kullanmazsan memory leak riski yüksek. Her subscription'da otomatik kullanma alışkanlığı edin.

afterNextRender / afterRender

DOM'a erişmek için modern API'lar. Eski ngAfterViewInit'in yerini aldı.

❏ Sorun: DOM'a Erişim

Bazen component render olduktan sonra gerçek DOM'a erişmek gerekir. Örneğin element'in genişliğini ölçmek, focus vermek, 3rd party library'yi başlatmak.

❏ afterNextRender — Bir Kez

```
import { Component, afterNextRender, ElementRef, ViewChild } from '@angular/core';

@Component({
  template: `<input #myInput type="text" />`
})
export class FocusComponent {
  inputEl = ViewChild<ElementRef<HTMLInputElement>>('myInput');

  constructor() {
    afterNextRender(() => {
      // İlk render TAMAMLANDI, DOM hazır
      this.inputEl()?.nativeElement.focus();
    });
  }
}
```

❏ afterRender — Her Render

afterRender her change detection cycle'da çalışır. Çok dikkatli kullan — performans kaybı.

```
constructor() {
  afterRender(() => {
    // Her render sonrası çalışır
    console.log('Render!');
  });
}
```

❏ Phases — Render Fazları

Render hook'ları farklı fazlarda çalışabilir:

- earlyRead: DOM'dan ölç ama yazma
- write: DOM'a yaz

- mixedReadWrite: İkisi de
- read: DOM'dan oku (write sonrası)

```
afterNextRender({
  earlyRead: () => {
    return this.element.nativeElement.scrollHeight; // Sadece oku
  },
  write: (scrollHeight) => {
    this.element.nativeElement.style.height = `${scrollHeight}px`; // Yaz
  },
});
```

Custom Attribute Directive

Direktif: HTML element'lerin davranışını değiştirmek için. Attribute directive en yaygın türü.

❑ Ne Zaman Custom Directive?

- Birden fazla yerde aynı davranış lazımsa (örn: hover effect)
- Component oluşturmak overkill ise
- 3rd party library'yi entegre etmek (jQuery plugin, vs.)
- Element'e custom event handling

❑ Basit Bir Directive — Highlight

```
import { Directive, ElementRef, HostListener, input } from '@angular/core';

@Directive({
  selector: '[appHighlight]', // [attribute] selector
})
export class HighlightDirective {
  // Input - directive'i parameterize et
  color = input<string>('yellow');

  private el = inject(ElementRef);

  @HostListener('mouseenter')
  onMouseEnter() {
    this.el.nativeElement.style.backgroundColor = this.color();
  }

  @HostListener('mouseleave')
  onMouseLeave() {
    this.el.nativeElement.style.backgroundColor = '';
  }
}
```

Kullanım:

```
<p appHighlight>Hover üzerime!</p>
<p appHighlight color="lightblue">Mavi hover</p>

<!-- Component'te import et -->
@Component({
  imports: [HighlightDirective],
  ...
})
```

□ host Property (Modern)

Modern Angular'da @HostListener ve @HostBinding yerine host property önerilir:

```
@Directive({
  selector: '[appHighlight]',
  host: {
    '(mouseenter)': 'onMouseEnter()',
    '(mouseleave)': 'onMouseLeave()',
    '[style.backgroundColor]': 'isHovered() ? color() : ""',
    '[class.active]': 'isActive()',
  },
})
export class HighlightDirective {
  color = input<string>('yellow');
  isHovered = signal(false);
  isActive = input<boolean>(false);

  onMouseEnter() { this.isHovered.set(true); }
  onMouseLeave() { this.isHovered.set(false); }
}
```

□ Pratik Örnekler

Örnek 1: Click Outside

```
@Directive({
  selector: '[clickOutside]',
  host: { '(document:click)': 'onDocClick($event)' }
})
export class ClickOutsideDirective {
  clickOutside = output<void>();
  private el = inject(ElementRef);

  onDocClick(event: Event) {
    if (!this.el.nativeElement.contains(event.target)) {
      this.clickOutside.emit();
    }
  }
}
```

Örnek 2: Tooltip

```

@Directive({
  selector: '[tooltip]',
  host: {
    '(mouseenter)': 'show()',
    '(mouseleave)': 'hide()',
  }
})
export class TooltipDirective {
  tooltip = input.required<string>();
  private el = inject(ElementRef);
  private tip?: HTMLElement;

  show() {
    this.tip = document.createElement('div');
    this.tip.textContent = this.tooltip();
    this.tip.className = 'tooltip';
    document.body.appendChild(this.tip);
    // Position logic...
  }

  hide() { this.tip?.remove(); }
}

```

Örnek 3: Auto Focus

```

@Directive({ selector: '[autoFocus]' })
export class AutoFocusDirective {
  private el = inject(ElementRef);

  constructor() {
    afterNextRender(() => {
      this.el.nativeElement.focus();
    });
  }
}

// Kullanım:
// <input autoFocus />

```

Built-in Directives — Modern Bakış

Eski Angular'da *ngIf, *ngFor, ngClass, ngStyle vardı. Modern Angular'da çoğu built-in syntax'la değişti.

❏ Eski vs Yeni — Karşılaştırma

Eski (Deprecated)	Modern (v17+)	Avantaj
*ngIf	@if / @else	Daha hızlı, type narrowing
*ngFor	@for + track	Daha hızlı, track zorunlu
*ngSwitch	@switch / @case	Daha temiz
ngClass	[class.x]	Daha basit
ngStyle	[style.x]	Daha basit

❏ @if — Modern Conditional

```
<!-- Basit if -->
@if (user) {
  <p>Welcome, {{ user.name }}</p>
}

<!-- if + else -->
@if (isLoggedIn()) {
  <app-dashboard />
} @else {
  <app-login />
}

<!-- if + else if + else -->
@if (status() === 'loading') {
  <app-spinner />
} @else if (status() === 'error') {
  <app-error [message]="error()" />
} @else {
  <app-content [data]="data()" />
}

<!-- Type narrowing (TypeScript bilir!) -->
@if (user(); as u) {
  <p>{{ u.name }}</p> <!-- u kesin User -->
}
```

❑ @for — Modern Loop

```
<!-- Track ZORUNLU -->
@for (item of items(); track item.id) {
  <app-item [data]="item" />
}

<!-- Index ile -->
@for (item of items(); track item.id; let i = $index) {
  <div>{{ i + 1 }}. {{ item.name }}</div>
}

<!-- Contextual variables -->
@for (item of items(); track item.id;
      let first = $first,
      let last = $last,
      let even = $even,
      let odd = $odd) {
  <div [class.first]="first" [class.last]="last">
    {{ item.name }}
  </div>
}

<!-- @empty: Liste boşsa -->
@for (item of items(); track item.id) {
  <app-item [data]="item" />
} @empty {
  <p>Henüz öge yok</p>
}
```

❑ @switch

```
@switch (userRole()) {
  @case ('admin') {
    <app-admin-panel />
  }
  @case ('user') {
    <app-user-dashboard />
  }
  @case ('guest') {
    <app-guest-view />
  }
  @default {
    <app-not-authorized />
  }
}
```

❑ @let — Template Variables (v18+)

```
<!-- Template'te local variable -->
@let fullName = user().firstName + ' ' + user().lastName;
@let isAdult = user().age >= 18;

<h1>{{ fullName }}</h1>
@if (isAdult) {
  <button>Yetişkin İçerik</button>
}
```

□ Class & Style Binding

```
<!-- Tek class -->
<div [class.active]="isActive()">

<!-- Birden fazla class (object) -->
<div [class]="{
  active: isActive(),
  disabled: isDisabled(),
  large: size() === 'lg'
}">

<!-- Tek style -->
<div [style.color]="textColor()">
<div [style.width.px]="width()">

<!-- Object style -->
<div [style]="{
  color: textColor(),
  fontSize: fontSize() + 'px',
}">
```


@defer — Lazy Loading

Angular 17+'da gelen @defer. Component'leri lazy yüklemek için template-level API.

❏ @defer Nedir?

@defer: "Bu kısmı şimdi değil, sonra yükle" demenin modern yolu. Çok büyük component'leri (chart, editor, vs.) ilk render'da yüklemezsin.

❏ Temel Kullanım

```
<!-- Default: Hemen yükler -->
<app-comments />

<!-- @defer: Lazy load -->
@defer {
  <app-comments />
}

<!-- @defer + placeholder + loading + error -->
@defer {
  <app-comments />
} @placeholder {
  <div>Yorumlar yüklenecek...</div>
} @loading (after 100ms; minimum 1s) {
  <app-spinner />
} @error {
  <p>Yorumlar yüklenemedi</p>
}
```

⚡ Triggers — Ne Zaman Yüklensin?

1. on idle (default)

```
@defer (on idle) {
  <app-comments />
}
<!-- Browser idle olduğunda yüklenir -->
```

2. on viewport

```
@defer (on viewport) {
  <app-heavy-chart />
}
<!-- Element viewport'a girince yüklenir (scroll edince) -->
```

3. on interaction

```
@defer (on interaction) {
  <app-comments />
} @placeholder {
  <button>Yorumları yükle</button>
}
<!-- Placeholder'a tıklayınca yüklenir -->
```

4. on hover

```
@defer (on hover) {
  <app-preview />
} @placeholder {
  <div>Önizle</div>
}
<!-- Hover yapınca yüklenir -->
```

5. on timer

```
@defer (on timer(3s)) {
  <app-banner />
}
<!-- 3 saniye sonra yüklenir -->
```

6. on immediate

```
@defer (on immediate) {
  <app-heavy />
}
<!-- Anında yüklenir (ama hala lazy bundle) -->
```

7. when (custom condition)

```
@defer (when shouldLoad()) {
  <app-content />
}
<!-- Signal true olunca yüklenir -->
```

□ Kombine Trigger'lar

```
<!-- Birden fazla trigger -->
@defer (on viewport; on hover; on timer(5s)) {
  <app-content />
}
<!-- Hangisi önce olursa, o tetikler -->

<!-- prefetch ile pre-load -->
@defer (on viewport; prefetch on idle) {
  <app-content />
}
<!-- Idle iken bundle indir, viewport'a girince göster -->
```

□ Best Practices

- ✓ Büyük 3rd party library'ler için kullan (chart, editor, video)
- ✓ Below the fold içerik için on viewport
- ✓ Modal content için on interaction
- ✓ @placeholder her zaman koy — UX iyi olur
- ✓ Bundle size analizi yap, gerçekten faydalı olduğundan emin ol
- ✗ Her component'i @defer'le sarma — overhead
- ✗ Critical content'i defer etme (üst banner, nav)
- ✗ Çok kısa timer'lar (1s'den az)
- ✗ @placeholder olmadan @defer (UX bozuk)

Modül 6 Özeti

Bu modülde Angular component'lerinin yaşam döngüsünü ve direktifleri öğrendik. Eski hook-based yaklaşımdan modern signal-based yaklaşıma geçişi gördük.

Neler Öğrendin?

- ✓ Component lifecycle aşamalarını
- ✓ constructor vs ngOnInit farkını
- ✓ Modern DestroyRef ve takeUntilDestroyed
- ✓ afterNextRender / afterRender API'larını
- ✓ Custom attribute directive yazmayı
- ✓ Built-in directive'lerin modern alternatiflerini
- ✓ @defer ile lazy loading
- ✓ Host bindings (host property)

Sıradaki Modül

Modül 7 — Services & Dependency Injection. Service patterns, inject() function, provider hierarchies, injection contexts, multi providers ve daha fazlası.